Word Embeddings & RNN

Physical Commonsense Reasoning on PIQA

NLP – Sacha Vogel

Preprocessing

1 **HTML Element Analysis**
No elements found, no cleaning necessary

2 **Tokenization**
NLTK tokenizer on lowercased text

3 **Stemming & Lemmatization**
Not applied because subword capability of fastText

4 **Punctuation/Stopwords Removal**
Punctuation removed – Stopwords are kept

5 **Truncation**
After analysis set length = 62

6 **Input Fields**
‘goal’ + <SEP> + ‘sol1’/’sol2’

7 **Vocabulary & Unknown Words**
Training split (vocabulary) / valid & test split (<UNK>)

8 **Encoding & Padding**
Word-to-index & filled with ‘<PAD>’ for tensor

Input / Output Format

input1
‘goal’ + <SEP> + ‘sol1’
[12, 34, 56, …] (62 token ids)

input2
‘goal’ + <SEP> + ‘sol2’
[12, 34, 78, …] (62 token ids)

X

embedding_matrix (15470, 300)

each unique token has ist own vector
(fastText ‘cc.en.300.bin‘)

[[98, 76, 54, …],
[191, 74, 534, …],
…]

- <PAD> = zero vector
- <UNK> = random vector
- <SEP> = random vector

=

embedded in both architectures
(62, 300)

vector dim 300 for each token

Input / Output Format – Architecture 1

pooling inputs:

1. pool goal – 300 dim vector
2. pool solution – 300 dim vector
3. concatenate – 2 * 300 dim vector = combined

compare:

- diff = combined1 - combined2

classify:

- classifier input = diff + combined1

output:

- 2 logits → argmax → label 0 or 1

Input / Output Format – Architecture 2

encoding inputs:

- LSTM reads embeddings left-to-right
  final hidden state = 128 dim vector
- ignore padding embeddings
- embeddings are trainable

concatenate:

- concatenate both hidden states = 2 * 128 = 256 dim vector

classify:

- classifier input = concatenated 256 dim vector

output:

- 2 logits → argmax → label 0 or 1

Network Architecture 1

1. fastText 300-dim embeddings, frozen

2. Separate masked mean pooling for goal and solution → 600-dim

3. Linear(600 → hidden) → ReLU → Dropout → Linear(hidden → 2)

Network Architecture 2

1. fastText 300-dim embeddings, trainable

2. 2-layer LSTM, padding tokens skipped via pack_padded_sequence

3. Final hidden state → Linear(2×hidden → hidden) → ReLU → Dropout → Linear(hidden → 2)

Experiments

1. Optimizer: AdamW, gradient clipping at 1.0
2. LR schedule: halved when validation accuracy stops improving (patience 2)
3. Early stopping: patience 5 epochs, max 30 epochs
4. Best model per architecture saved by validation accuracy
5. Bayesian hyperparameter sweep (20 runs per architecture)

Experiments

5 Bayesian hyperparameter sweep (20 runs per architecture)

hyperparameter	**values**
learning	1e-3, 1e-4, 1e-5
weight decay	1e-3, 1e-4, 1e-5
hidden dim	128, 256, 512
dropout probability	0.1, 0.3, 0.5

Results

	loss	accuracy
Embedding Classifier	0.7058	0.563
RNN Classifier	4.69834	0.576

Interpretation

RNN trains significantly slower
more parameters, sequential computation

RNN outperforms embedding classifier
word order matters for commonsense

Most impactful decisions

- separate goal/solution pooling
- pack_padded_sequence

even large models struggle without physical world experience

Thank You For Your Attention!

Are There Any Questions?